

프로그래밍 역량 강화 전문기관, 민코딩

Unit Test의 이해



목차

1. Unit Test 개요
2. Maven 프로젝트 생성
3. JUnit 설치 및 테스트
4. Unit Test 의미와 작성 원칙

Unit Test 개요

OOP 에서 말하는 모듈이란?

유례

- 큰 시스템을 작은 조각의 문제로 나누어 생각할 때, 작은 조각을 '모듈' 이라고 한다.

모듈의 정의

- 독립적으로 **기능**을 수행할 수 있는 작은 단위
- 한가지 일을 맡으며, 독립적이고 단순해야 한다.
- 일반적으로 함수 or 클래스 하나를 의미

Unit Test

- ✓ 모듈 단위로, Testing을 하는 것
입력값을 넣어 보았을 때,
결과값이 기대값과 동일하게 나오는지 확인한다.
- ✓ 주로 검증팀보다는, 개발팀에서 Test를 수행



Unit Test 프레임워크

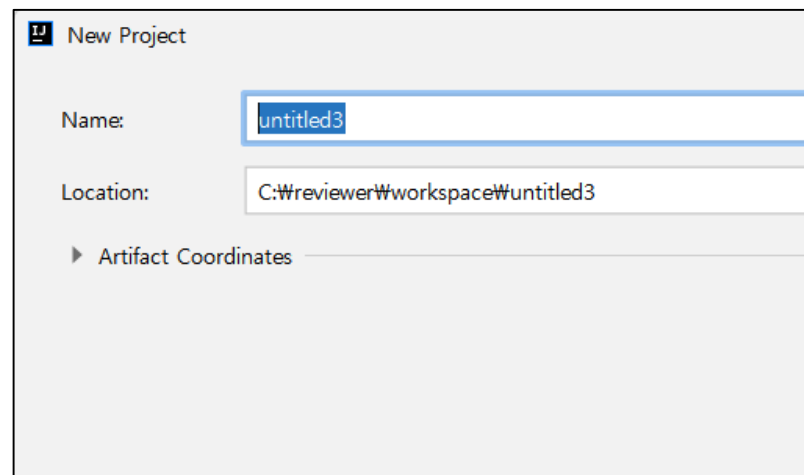
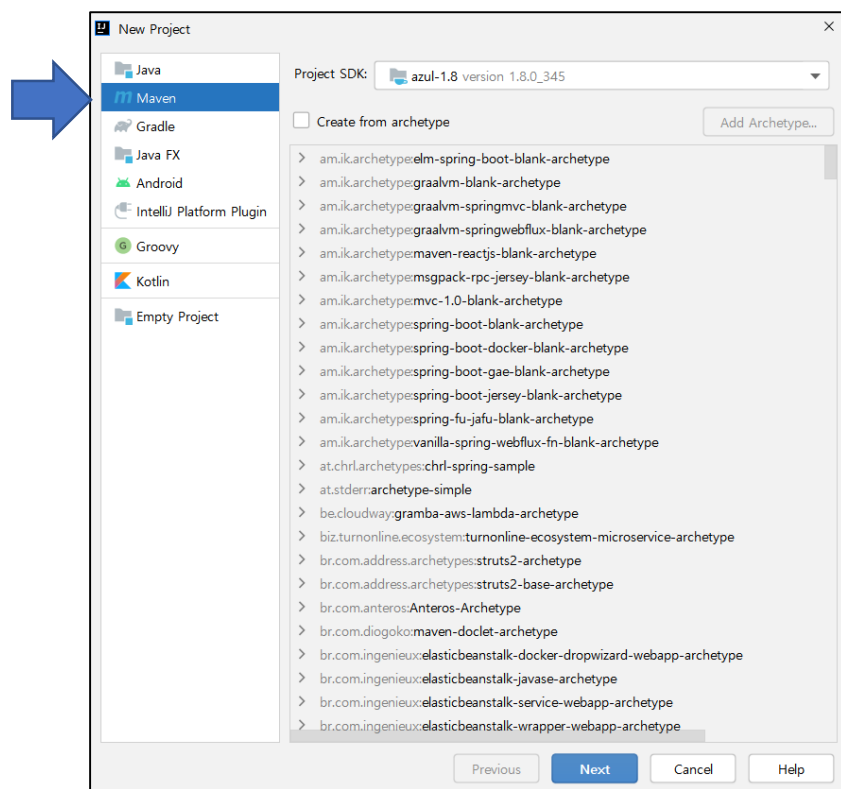
가장 많이 사용되는 Unit Test 프레임워크

1. Java : **Junit**
2. C++ : **Google Test**
3. Python : **unittest** (Python 내장)

Maven 프로젝트 생성

Maven을 이용한 Unit Test 설치 방법

먼저 New 프로젝트 생성



프로젝트 명 기입 후 Finish 버튼

Maven 빌드도구를 사용하는 프로젝트 생성
메뉴경로 (File > New > Project)

[참고] Maven

Java의 대표적인 Build 자동화 도구

- IDE 독립적으로 빌드 환경을 구성할 수 있다.
 - Maven 빌드 도구를 사용하면
Eclipse / IntelliJ 무엇으로 개발하던 상관없게 된다.
- 프로젝트에 필요한 Library 설치가 편리하다.
 - Dependency 파일들을 자동으로 설치해준다.

폴더 구조

.idea

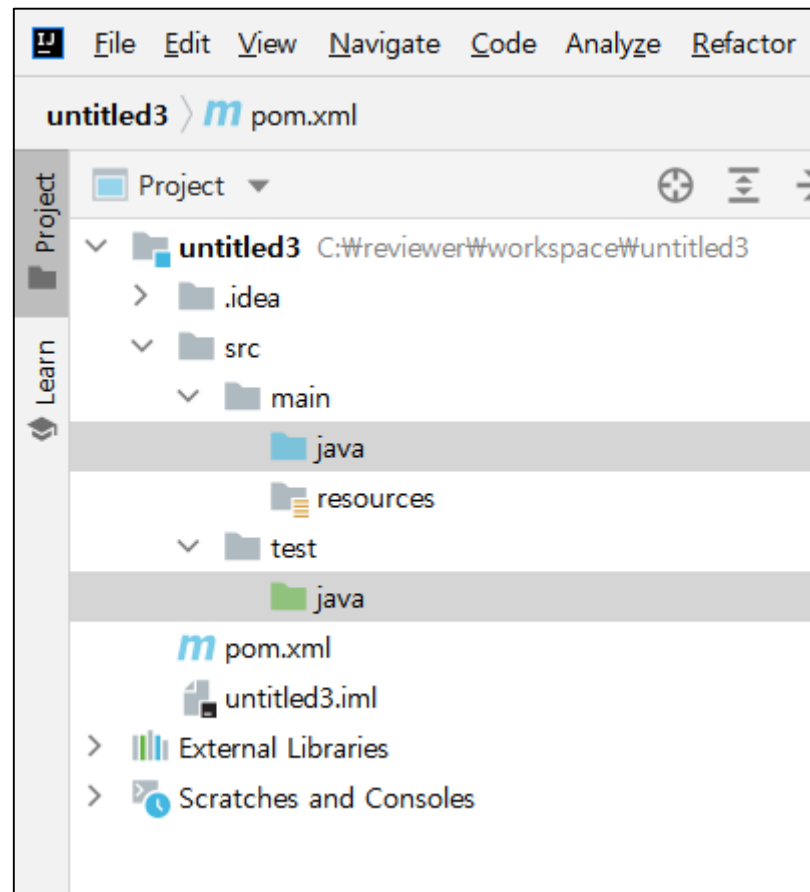
- IntelliJ전용 폴더

src > main

- 개발할 소스코드
- (resource : 소스코드 외 나머지들)

src > test

- 테스트 코드 넣는 곳



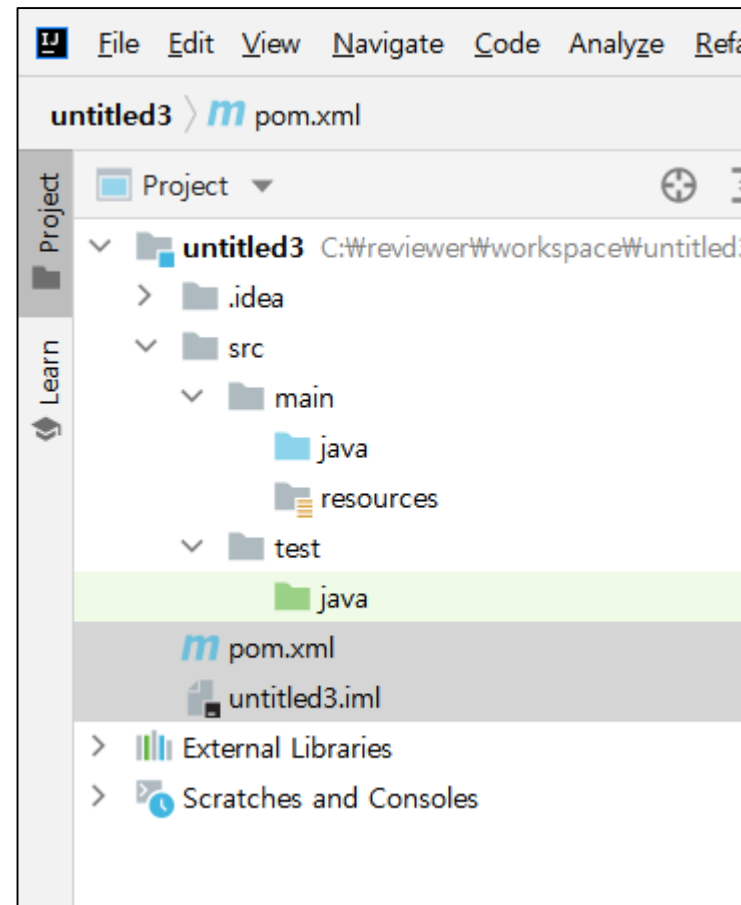
Maven 파일

pom.xml

- Maven 설정파일
- 이곳에 설치할 Library를 기록하고
(Dependency 쪽에 설치하고픈 Library 내용 추가)
새로고침하면 자동 설치가 시작된다.

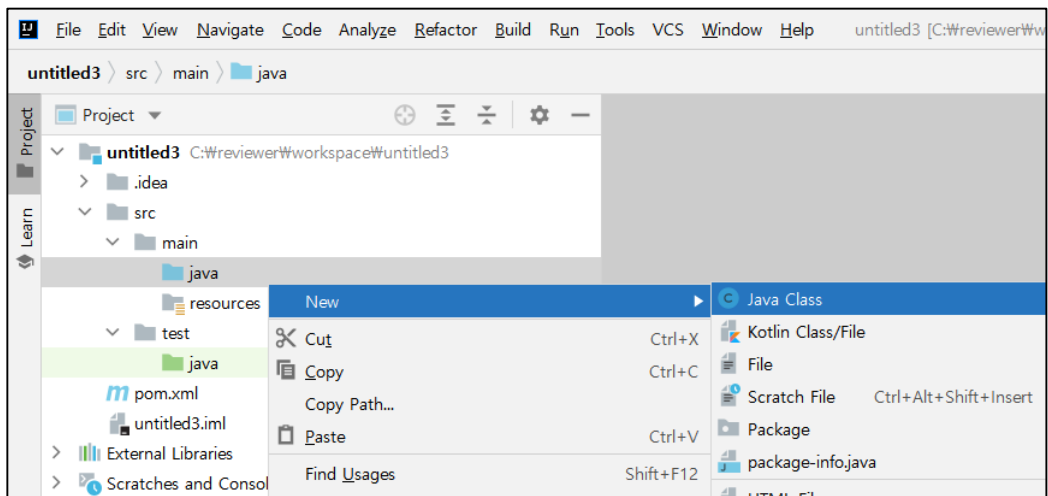
*.iml 파일

- Maven 동작 시
Dependency 관련 내용이 자동 기입되는 파일
- Maven의 metadata (우리가 건들지 않음)

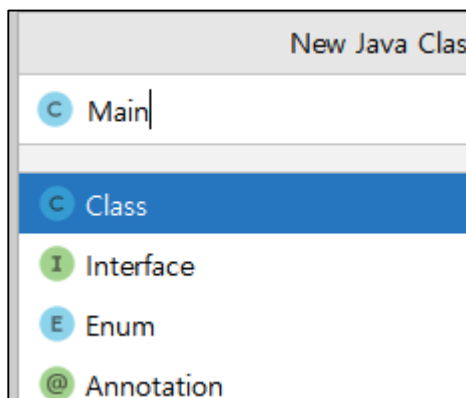


Hello World 부터

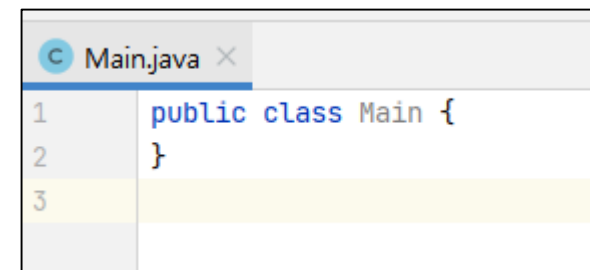
Main.java 파일 추가



src > main에 파일 추가하기



파일명 입력



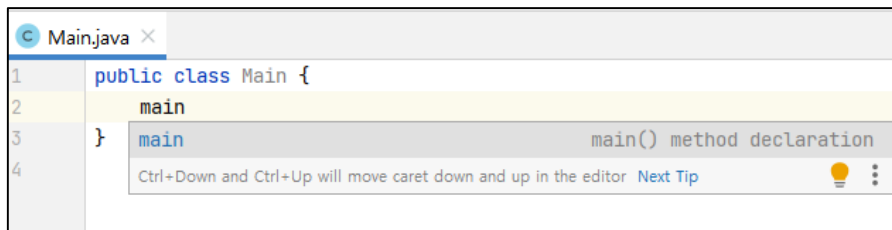
생성완료

자동완성 기능

두 가지 방법을 모두 해볼 것!

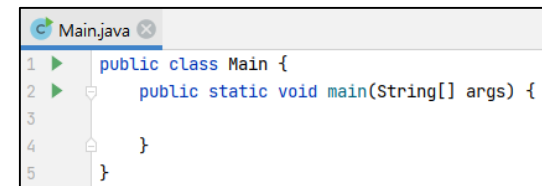
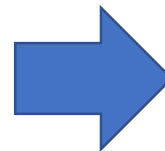
- 단축키 암기 : Ctrl + Space

시도 1



```
1 public class Main {
2     main
3 }
```

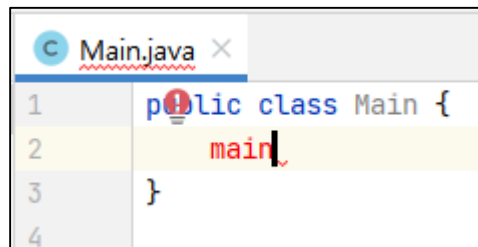
main 이라고 입력 후 [Tab] 키



```
1 public class Main {
2     public static void main(String[] args) {
3
4     }
5 }
```

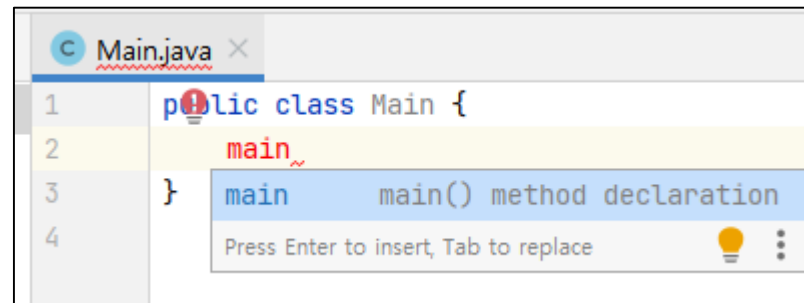
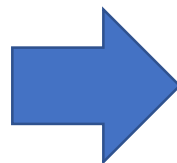
자동완성이 된다.

시도 2



```
1 public class Main {
2     main
3 }
```

main 입력 후 Ctrl + Space

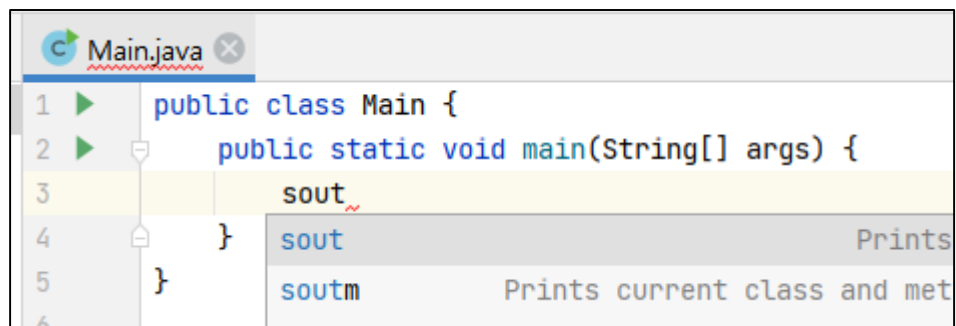


```
1 public class Main {
2     main
3 }
```

[Tab] 키를 누른다.

Console Log 출력

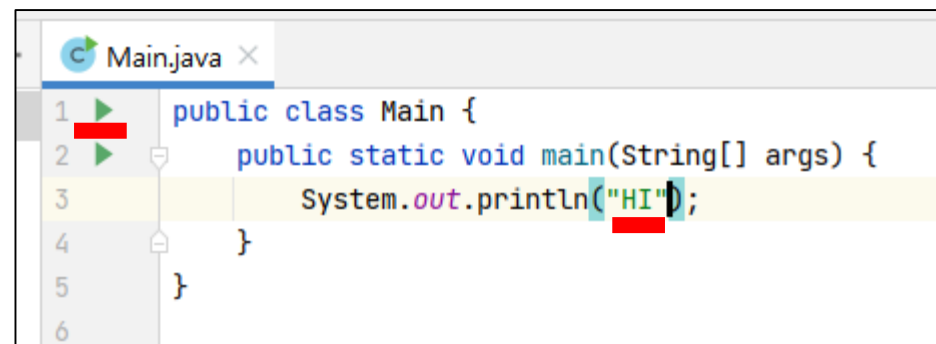
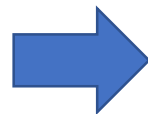
콘솔 출력



```
1 public class Main {
2     public static void main(String[] args) {
3         sout
4     }
5 }
```

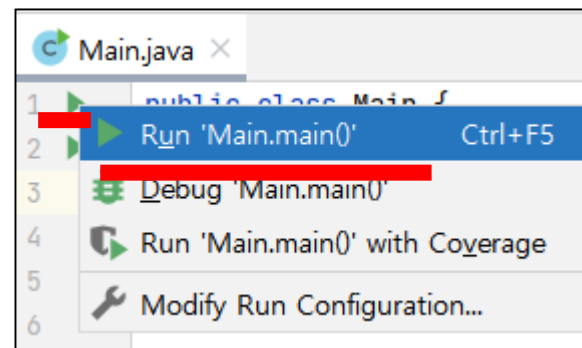
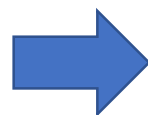
sout Prints current class and met

sout 입력 후 Tab 키
(또는 Ctrl + Space 누르고 Tab)



```
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println('HI');
4     }
5 }
```

HI 추가 입력



```
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println('HI');
4     }
5 }
```

- Run 'Main.main()' Ctrl+F5
- Debug 'Main.main()'
- Run 'Main.main()' with Coverage
- Modify Run Configuration...

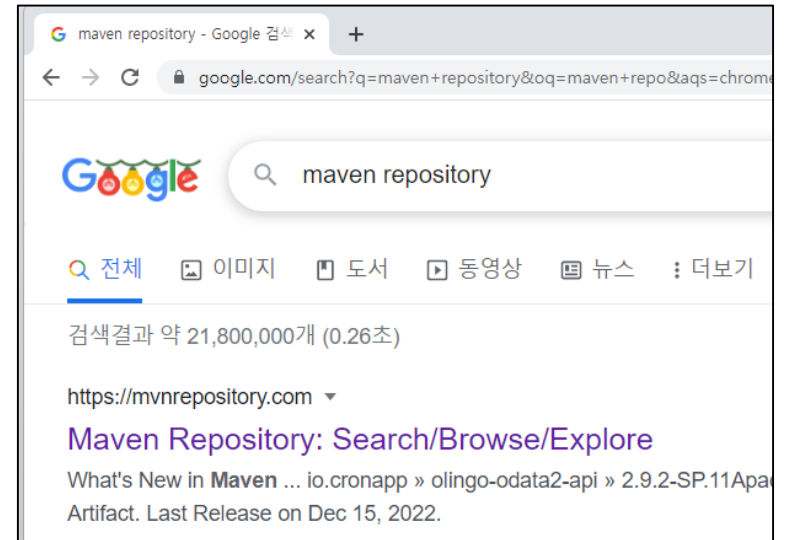
JUnit 설치 및 테스트

Maven Repository

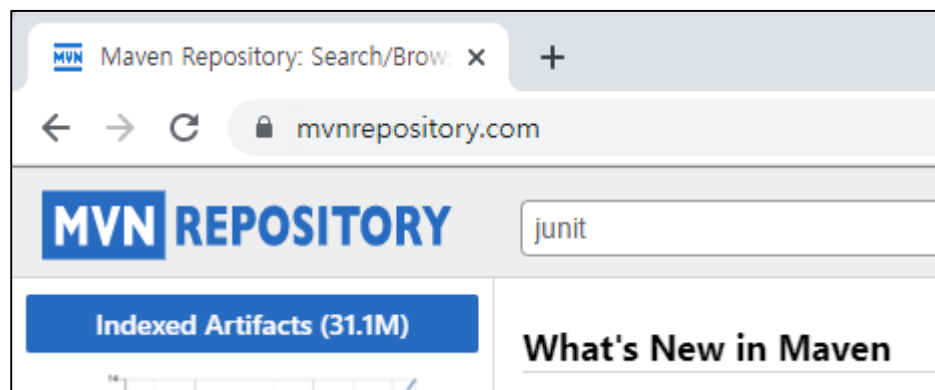
Maven 도구를 사용하여
JUnit이라는 Library를 설치하고자 한다.

이를 위해 먼저,
maven repository 구글링하여 사이트에 접속한다.

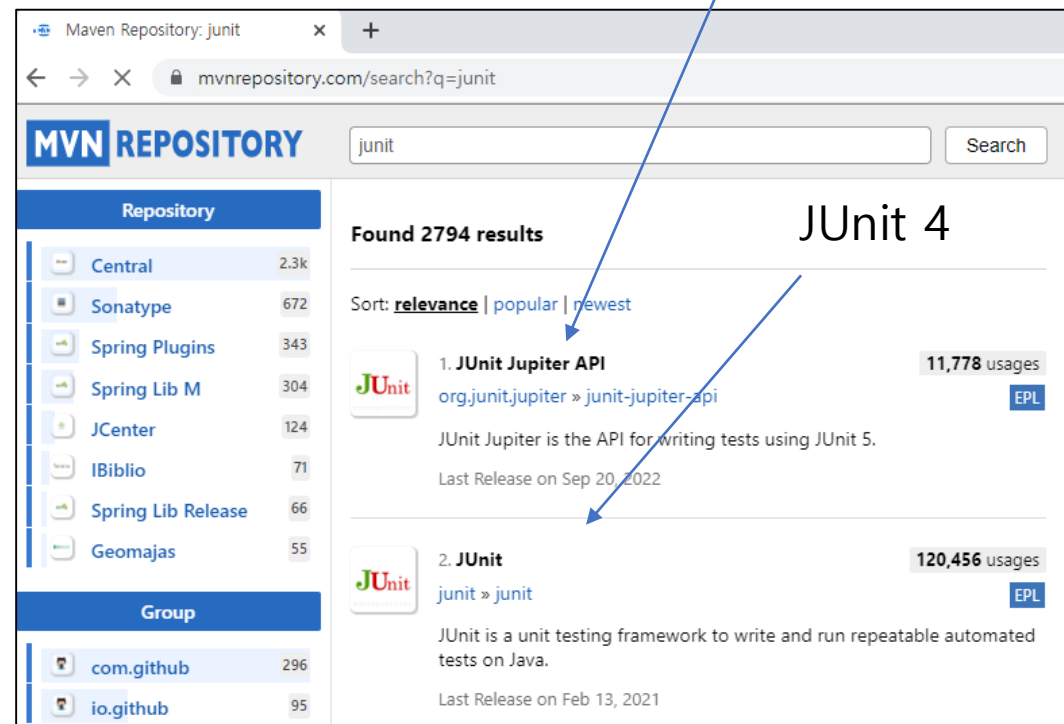
- <https://mvnrepository.com/>



JUnit 검색

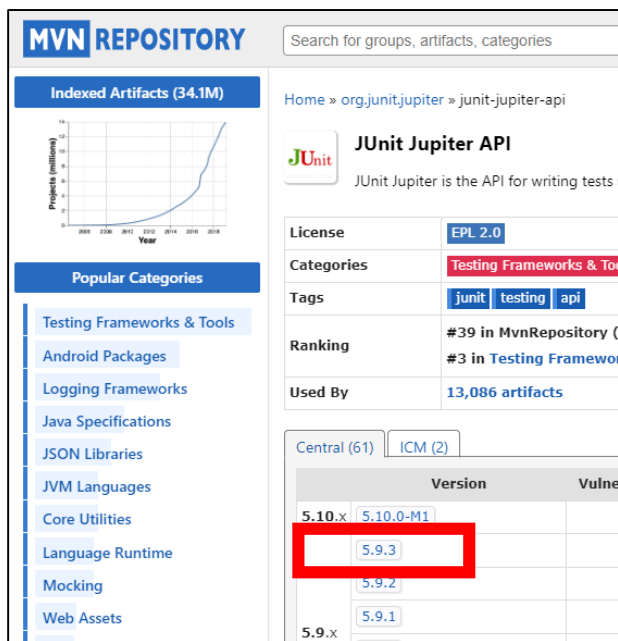


JUnit 이라고 검색어 입력 후 검색



JUnit 5.x 버전 선택

최상단의 5.x 버전 선택
(실험 버전인 M 버전 제외)



The screenshot shows the Maven Repository page for the JUnit Jupiter API. The '5.10.x' section is expanded, and the '5.9.3' version is highlighted with a red box. The page also shows the '5.9.x' section with versions 5.9.1 and 5.9.2.

5.x 버전의 최신 버전 선택



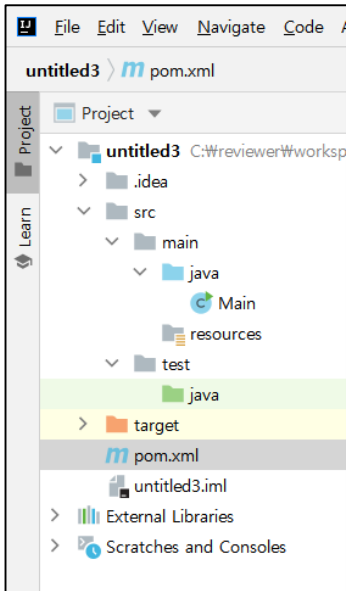
The screenshot shows the JUnit Jupiter API page on the Maven Repository. The '5.9.3' version is selected, and the 'Maven' tab is active, showing the dependency XML snippet:

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.9.3</version>
  <scope>test</scope>
</dependency>
```

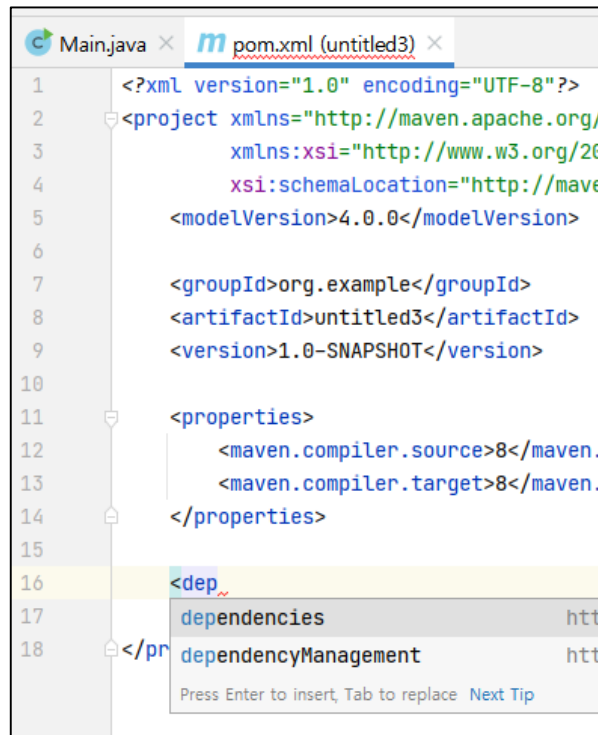
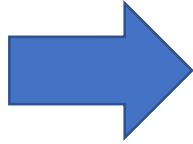
이 Text를 Maven에 입력하면, 자동설치가 된다.
Ctrl + C 로 내용을 복사하자.

pom.xml (1/2)

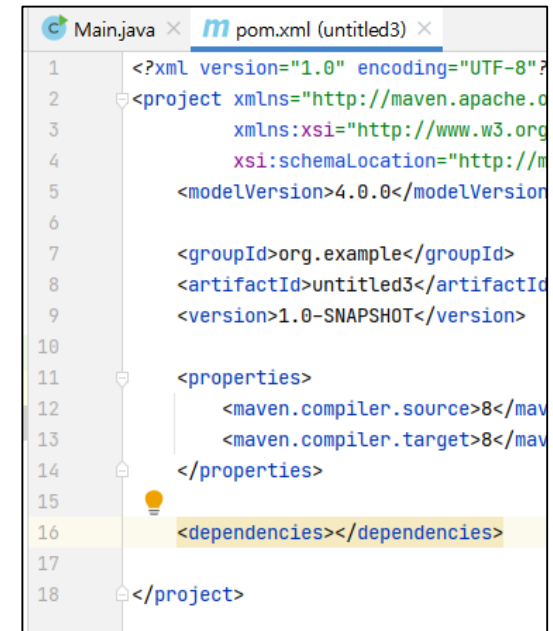
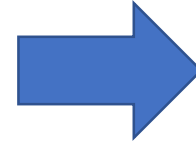
Library는 이 파일에다가
복사한 내용을 붙여넣기 하고, 새로고침하면 자동 설치가 시작된다.



pom.xml 파일 열기



"<dep" 입력 후 Tab키



자동완성 성공

pom.xml (2/2)

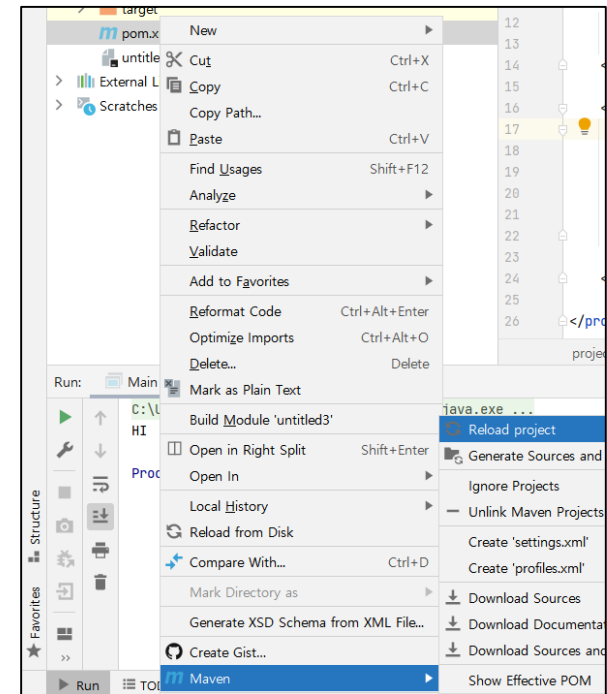
Library는 이 파일에다가
복사한 내용을 붙여넣기 하고, 새로고침하면 자동 설치가 시작된다.

```
6      <dependencies>
7
8    </dependencies>
9
10   </project>
```

엔터 입력 후
Ctrl + V (붙여넣기)

```
11   <properties>
12     <maven.compiler.source>14</maven.compiler.s
13     <maven.compiler.target>14</maven.compiler.t
14     <project.build.sourceEncoding>UTF-8</project
15   </properties>
16
17   <dependencies>
18     <dependency>
19       <groupId>org.junit.jupiter</groupId>
20       <artifactId>junit-jupiter-api</artifact
21       <version>5.9.3</version>
22       <scope>test</scope>
23     </dependency>
24   </dependencies>
25
26 </project>
```

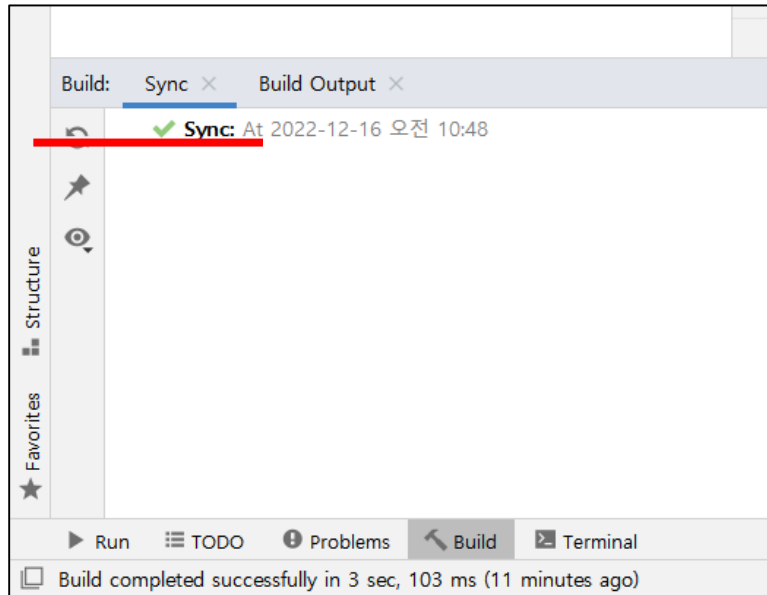
붙여넣기 완료
이제 새로고침 해야한다.



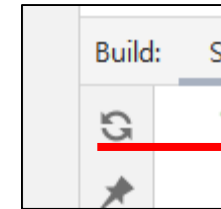
pom.xml 파일
오른쪽 버튼 > Maven > Reload

화면 하단, Build 탭 보기

Build 탭에 Sync가 뜨면 설치 완료



설치가 완료된 화면

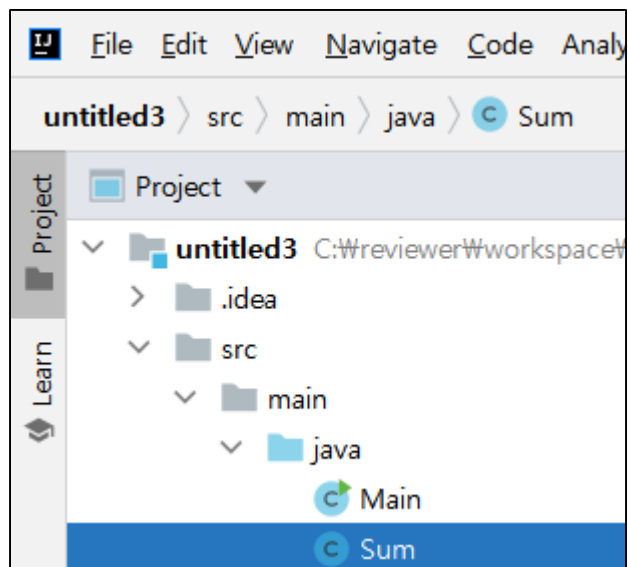


새로고침 버튼
(Reload project 버튼)

한번 눌러보자.

테스트 대상이 되는 파일 생성

Sum Class 생성 후
소스코드 삽입



Sum Class 추가

```
1 public class Sum {  
2     public int getSum(int t1, int t2) {  
3         return t1 + t2;  
4     }  
5 }
```

getSum 모듈 개발하기

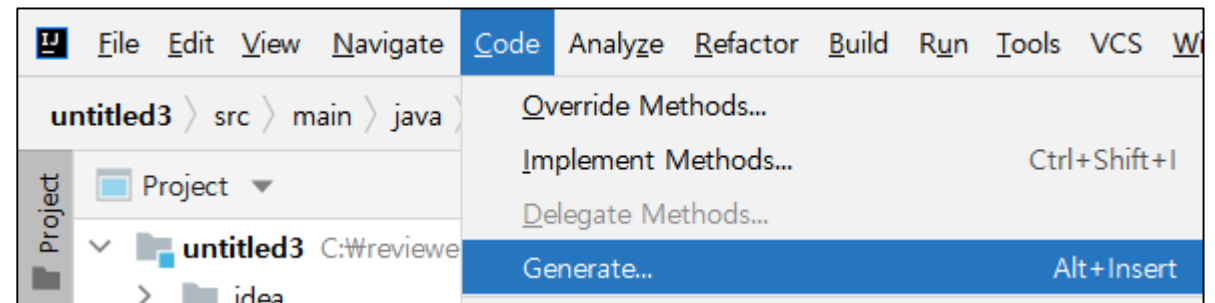
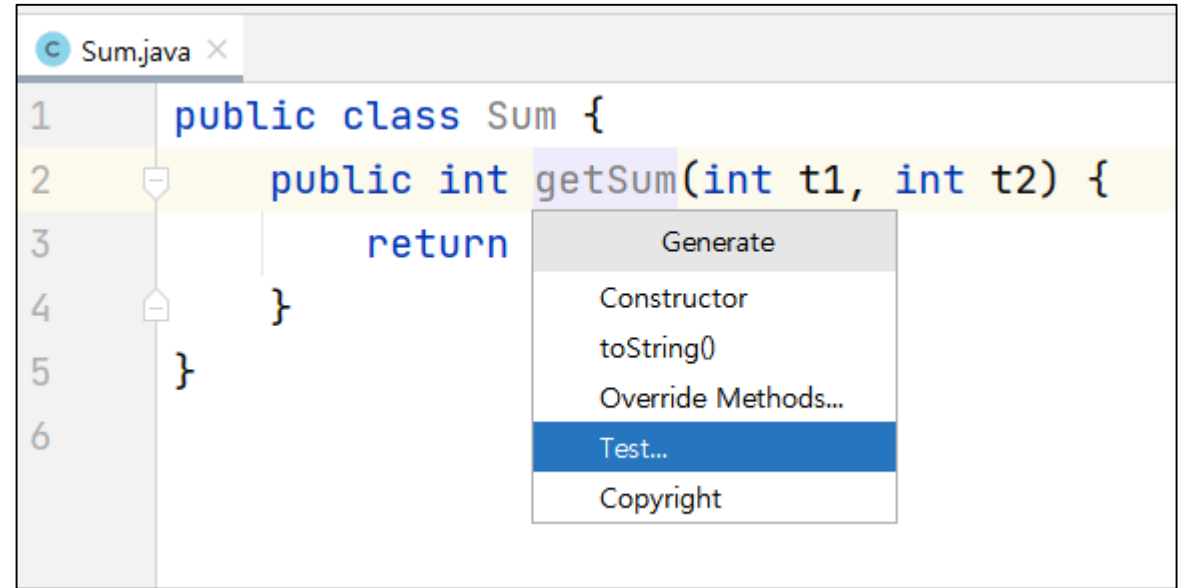
Generate

커서를 Sum Class 안에 두고,

Alt + Insert (단축키 암기!)

누르고

Test 선택하기

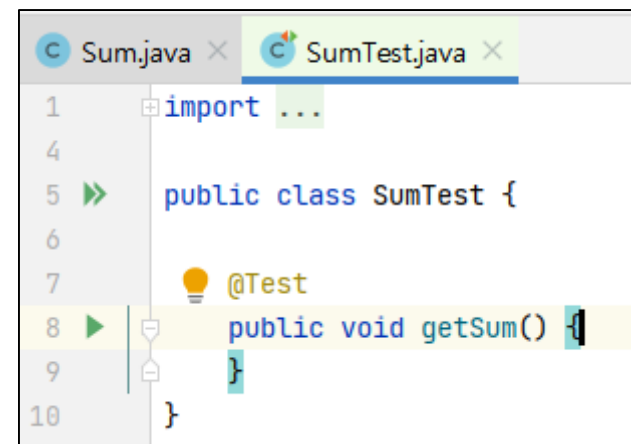
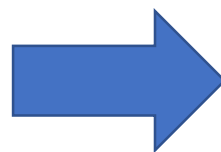
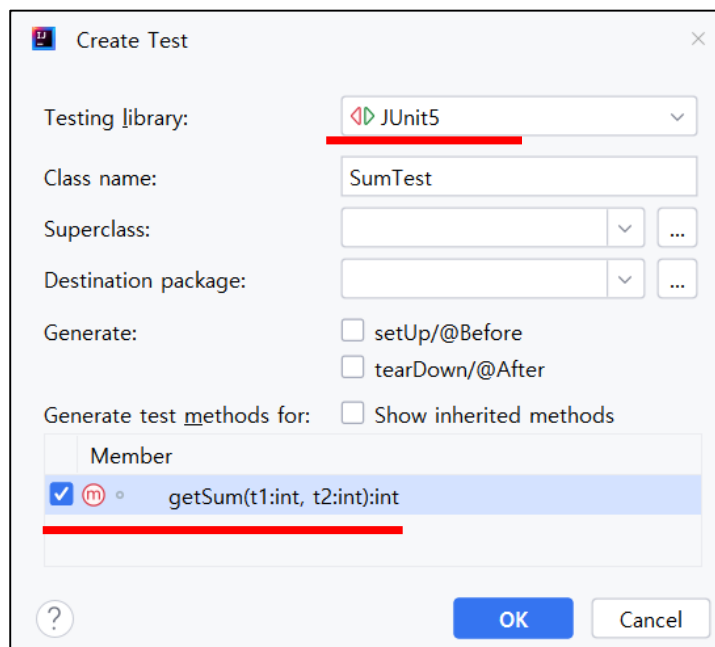


단축키 Alt + Insert 누르기. (암기 권장!)
또는 Code > Generate 선택해도 된다.

테스트 생성하기

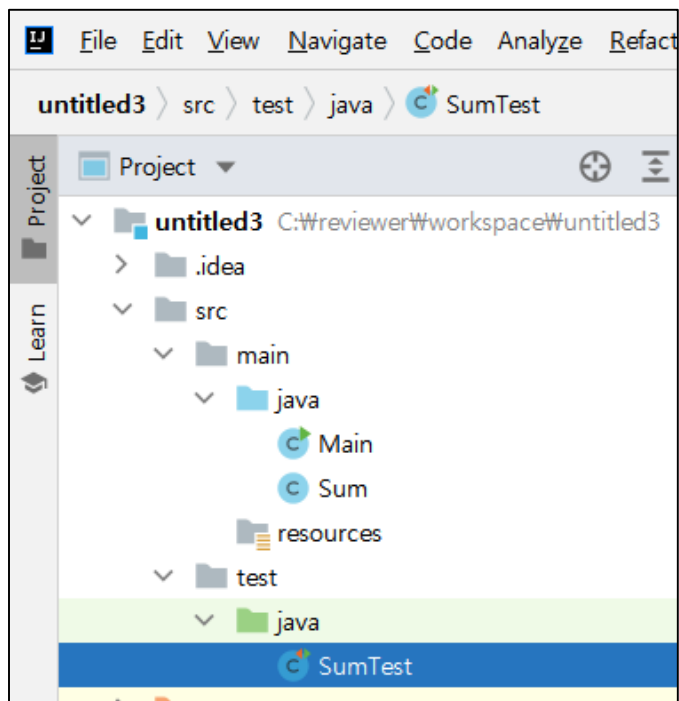
JUnit 5 선택

테스트할 함수명 선택하기

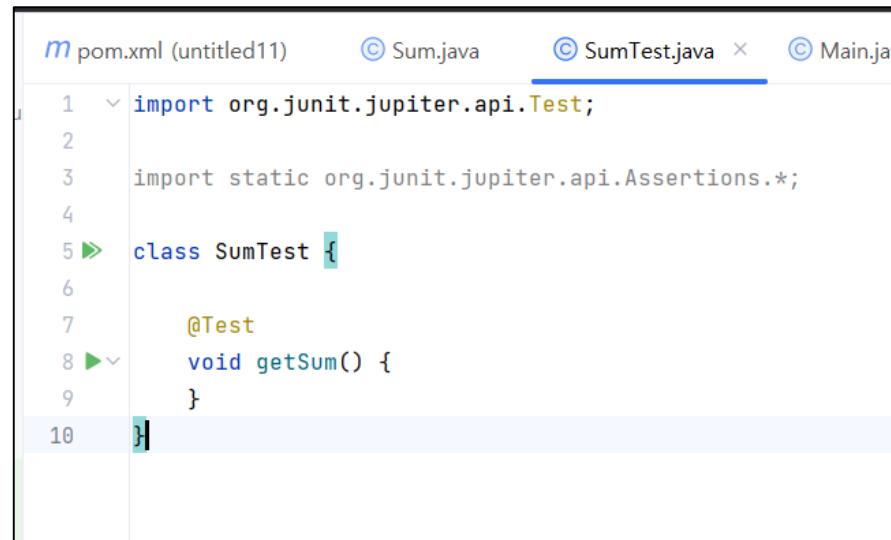


Test 파일 생성 완료

테스트 폴더를 보면, 자동생성 되어있다.



테스트할 파일 + Test 이름으로 생성됨

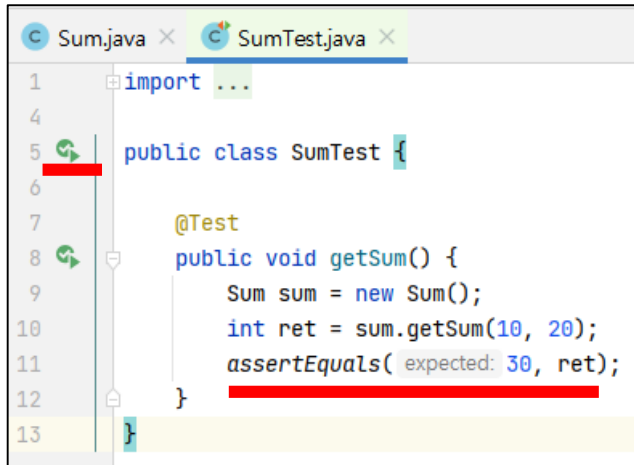


import 문이 자동 생성되어,
직접 추가하지 않고 편리하다.

테스트 코드 기입하기

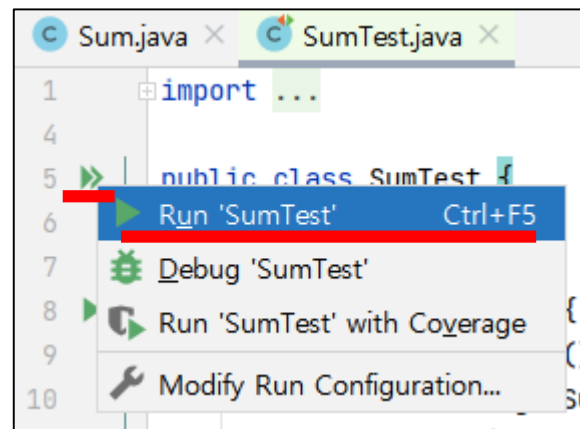
테스트 결과를 확인한다.

- 기대값을 일부러 틀려, Test에 Fail도 만들어보자.



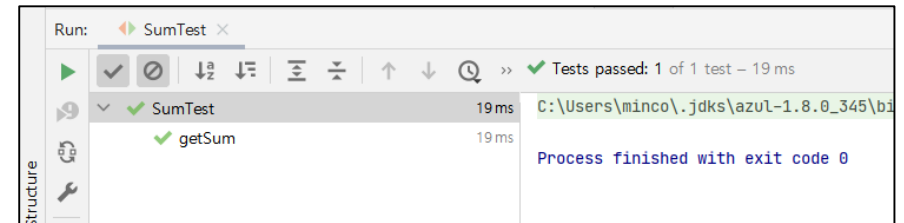
```
1 import ...
4
5 public class SumTest {
6
7     @Test
8     public void getSum() {
9         Sum sum = new Sum();
10        int ret = sum.getSum(10, 20);
11        assertEquals( expected: 30, ret);
12    }
13 }
```

ret가 30 이어야만 한다.



```
1 import ...
4
5 public class SumTest {
6
7     @Test
8     public void getSum() {
9         Sum sum = new Sum();
10        int ret = sum.getSum(10, 20);
11        assertEquals( expected: 30, ret);
12    }
13 }
```

Test 버튼 누르기



```
Run: SumTest
Tests passed: 1 of 1 test - 19 ms
SumTest
  getSum
    Process finished with exit code 0
```

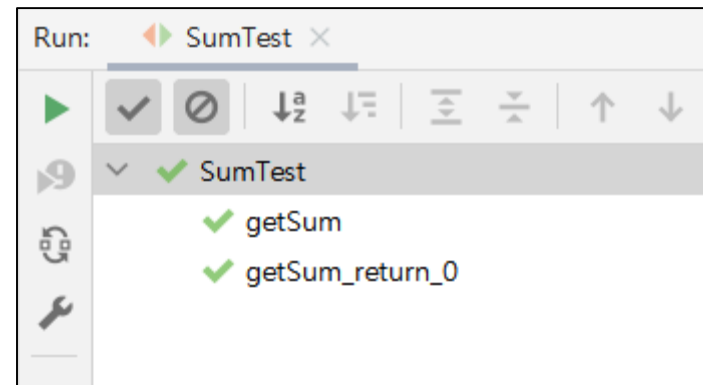
Test 결과 성공

테스트 케이스 하나 더 추가.

테스트 케이스 추가하기

- @Test 라는 '어노테이션' 을 매번 추가해준다.

```
public class SumTest {  
  
    @Test  
    public void getSum() {  
        Sum sum = new Sum();  
        int ret = sum.getSum(10, 20);  
        assertEquals( expected: 30, ret);  
    }  
  
    @Test  
    public void getSum_return_0() {  
        Sum sum = new Sum();  
        int ret = sum.getSum(+50, -50);  
        assertEquals( expected: 0, ret);  
    }  
}
```



[도전] 새 프로젝트 부터, 처음부터 만들기

Gop 모듈 생성하기

- 두 수의 곱을 리턴해주는 모듈

Unit Test하기

- Gop 모듈이 정상 동작하는지 테스트
- 테스트 케이스를 2개 만든다.

최대한 힌트를 보지 않고,
직접 작성해본다.

Assertion

실습 준비

Test 소스코드 준비

```
import ...

public class SumTest {

    @Test
    public void assertPractice() {

    }

}
```

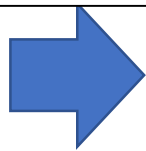
에러 메시지 추가하기

assertEquals (Fail 메시지, Expected, Actual)

- Expected : 기대값
- Actual : 실제값

```
@Test
void 에러메세지출력() {
    int actual = 1;
    int expected = 2;

    assertEquals(expected, actual, "Wrong, It's not same value");
}
```



Run CalTest.에러메세지출력 x

CalTest 9 ms

에러메세지출력() 9 ms

Tests failed: 1 of 1 test - 9 ms

org.opentest4j.AssertionFailedError: Wrong, It's not same value ==>
Expected :2
Actual :1
[<Click to see difference>](#)

assertTrue / assertFalse

- ✓ assertTrue : 조건식이 참 일때만 Pass
- ✓ assertFalse : 조건식이 거짓 일때만 Pass

```
@Test
void true_false() {
    int ret = 10;

    assertTrue(ret >= 5);
}
```

대소비교시 assertTrue 사용

```
@Test
void true_false() {
    int ret = 10;

    assertTrue(ret == 10);
}
```



```
@Test
void true_false() {
    int ret = 10;

    assertEquals(10, ret);
}
```

같은지 비교는 assertTrue 보다는
assertEquals를 사용하는 것이 가독성이 더 좋다.

배열 비교하기

ArrayList 는 assertEquals로 비교
array는 assertEquals로 비교

```
@Test
public void assertPractice() {
    ArrayList<Integer> arr1 = new ArrayList<>();
    ArrayList<Integer> arr2 = new ArrayList<>();

    arr1.add(1);
    arr1.add(2);
    arr2.add(1);
    arr2.add(2);

    assertEquals(arr1, arr2);
}
```

```
@Test
public void assertPractice() {
    int[] arr1 = {1, 2, 3, 4};
    int[] arr2 = {1, 2, 3, 4};

    assertEquals(arr1, arr2);
}
```

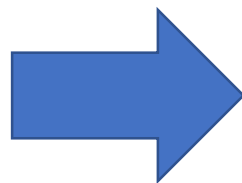
Null인지 테스트

Null인지 아닌지 테스트하기

```
@Test  
void loop_test() {  
  
    assertNull(actual: null);  
    assertNotNull(actual: null);  
  
}
```

무조건 Fail 발생

```
@Test  
public void assertPractice() {  
    fail("무조건 Fail 입니다");  
}
```



```
✖ Tests failed: 1 of 1 test – 0 ms  
C:\Users\minco\.jdk\azul-1.8.0_345\bin\java.exe ...  
java.lang.AssertionError: 무조건 Fail 입니다  
    <1 internal call>  
    at SumTest.assertPractice(SumTest.java:13) <25 internal calls>  
  
Process finished with exit code -1
```

[참고] assertEquals vs assertEquals

assertEquals

- 값을 비교하는 것.
- 매우 많이 사용된다.

assertEquals

- 참조값을 비교하는 것.
- 사용할 일이 적다.

Exception 테스트 1

Exception이 발생하면
PASS가 되는 테스트

```
@Test
void NullPointerException() {
    assertThrows(NullPointerException.class, () -> {
        ArrayList<Integer> arr = null;
        int ret = arr.get(0);
    });
}
```

JUnit 5 에서 Exception 처리 방법

get(0) 에서
Exception 발생하여 **Pass**

```
@Test(expected = IndexOutOfBoundsException.class)
public void assertPractice() {

    ArrayList<Integer> arr = new ArrayList<>();
    //int ret = arr.get(0);
}
```

[참고] **JUnit 4** 에서 Exception 방식

[참고] Exception 테스트 2

Exception이 발생하지 않으면 PASS

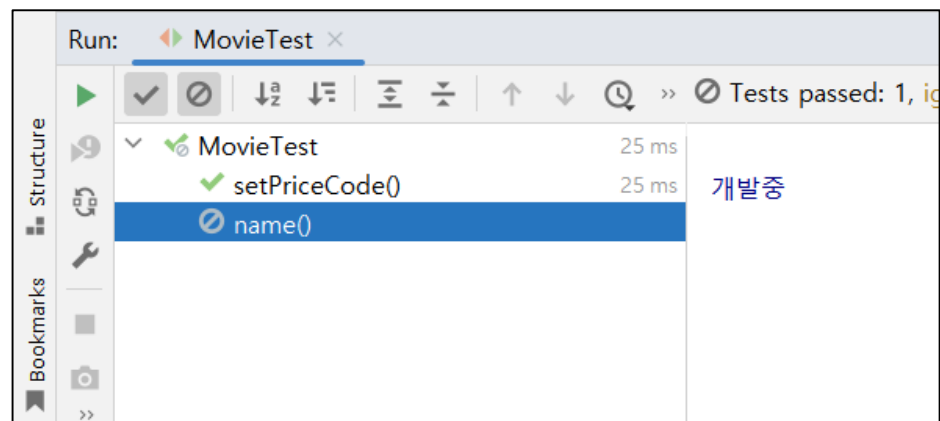
```
@Test
void NullPointerException() {
    assertDoesNotThrow(() -> {
        ArrayList<String> arr = new ArrayList<>();
        arr.add("HI");
        System.out.println(arr.get(0));
    });
}
```

@Disabled

특정 테스트만 진행을 생략하고자 할 때,
@Disabled 어노테이션을 추가한다.

```
@Test
@Disabled("개발중")
void name() {

}
```



[참고] @Timeout

지정된 ms 시간을 초과하면 Fail 발생

```
@Test
```

```
@Timeout(value = 10, unit = TimeUnit.MILLISECONDS)
```

```
void loop_test() {
```

```
    int sum = 0;
```

```
    for (int i = 0; i < 1000000000; i++) {
```

```
        sum = sum % 10 + i;
```

```
    }
```

```
    assertEquals(sum, actual: 1000000000);
```

```
}
```

10 ms 타임아웃 설정

Tests failed: 1 of 1 test - 1 sec 10 ms

C:\Users\inho\.jdk\azul-11.0.18\bin\java.exe ...

java.util.concurrent.TimeoutException: loop_test() timed out after 10 milliseconds

JUnit5 Life Cycle

✓ 각 TestCase가 시작하기 전, 후

- SetUp
- TearDown

```
@BeforeEach
void setUp() {

}

@AfterEach
void tearDown() {

}
```

```
@BeforeAll
static void beforeAll() {

}

@AfterAll
static void afterAll() {

}
```

✓ Unit Test 수행 전 1회성 수행

- BeforeClass
- After

✓ 유의사항

- BeforeClass는 생성자가 아니다.
필드 초기화는 Test Class 생성자에서 해야한다.

Generate	
Test Method	>
SetUp Method	>
TearDown Method	>
BeforeClass Method	>
AfterClass Method	>
Constructor	
toString()	
Override Methods...	
Copyright	

[도전] 훌쩍 Unit Test

ArrayList 에 숫자 값을 넣으면,

짝수면 "O" / 홀수면 "X"를 구분해주는 ArrayList를 리턴하는 모듈

- 입력예시 : {1, 2, 3, 0}
- 출력예시 : {"X", "O", "X", "O"}
- 모두 짝수이거나 홀수면 null 리턴

시작 코드만 만들고,
테스트 코드를 만들자.

```
import java.util.ArrayList;

public class OddEven {
    public ArrayList<String> getResult(ArrayList<Integer> nums) {
        ArrayList<String> result = new ArrayList<>();
        return result;
    }
}
```

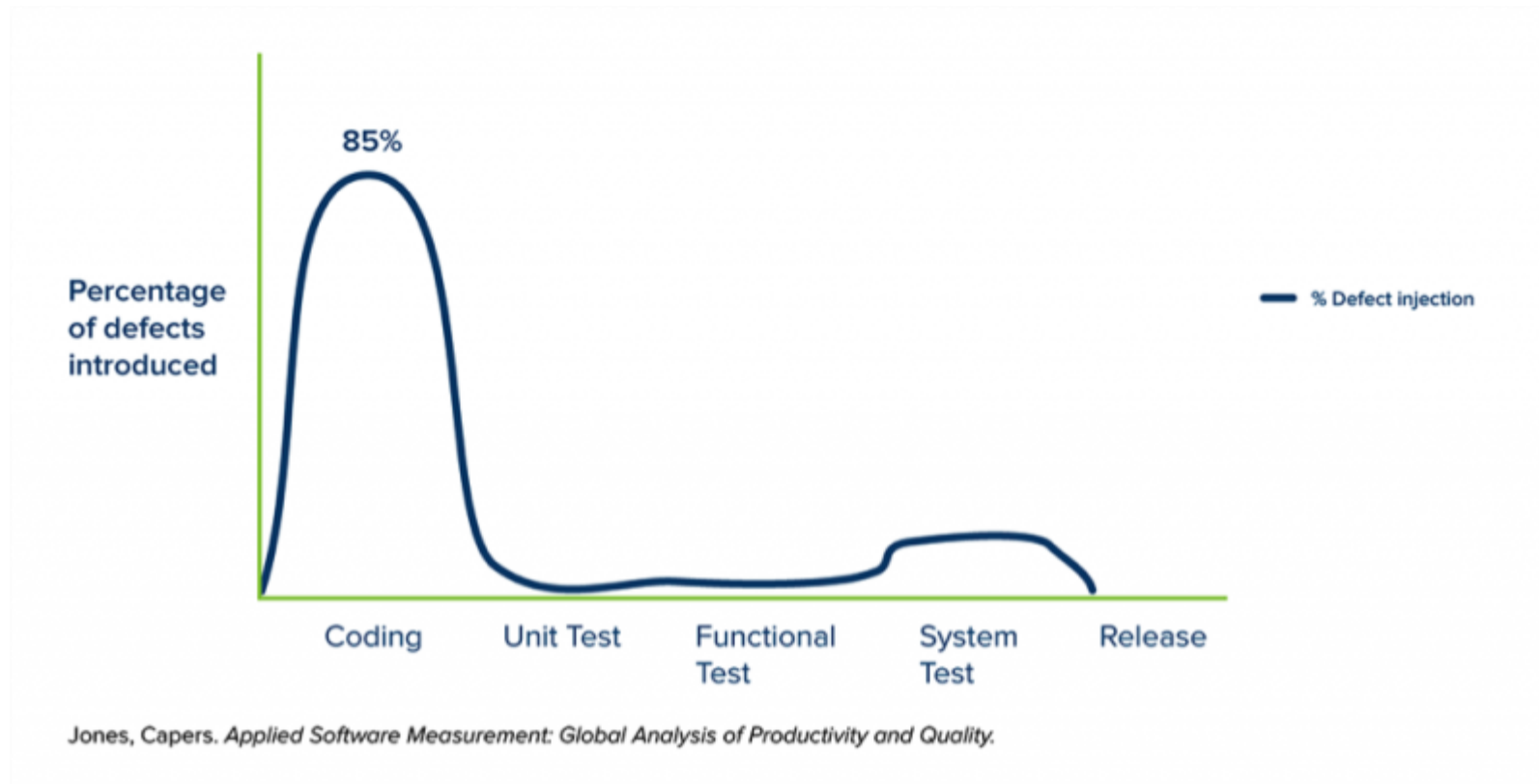
시작코드

Unit Test 의미와 작성 원칙

Unit Test 의 의미 : 결함의 비용 최소화

✓ 결함이 만들어지는 타이밍

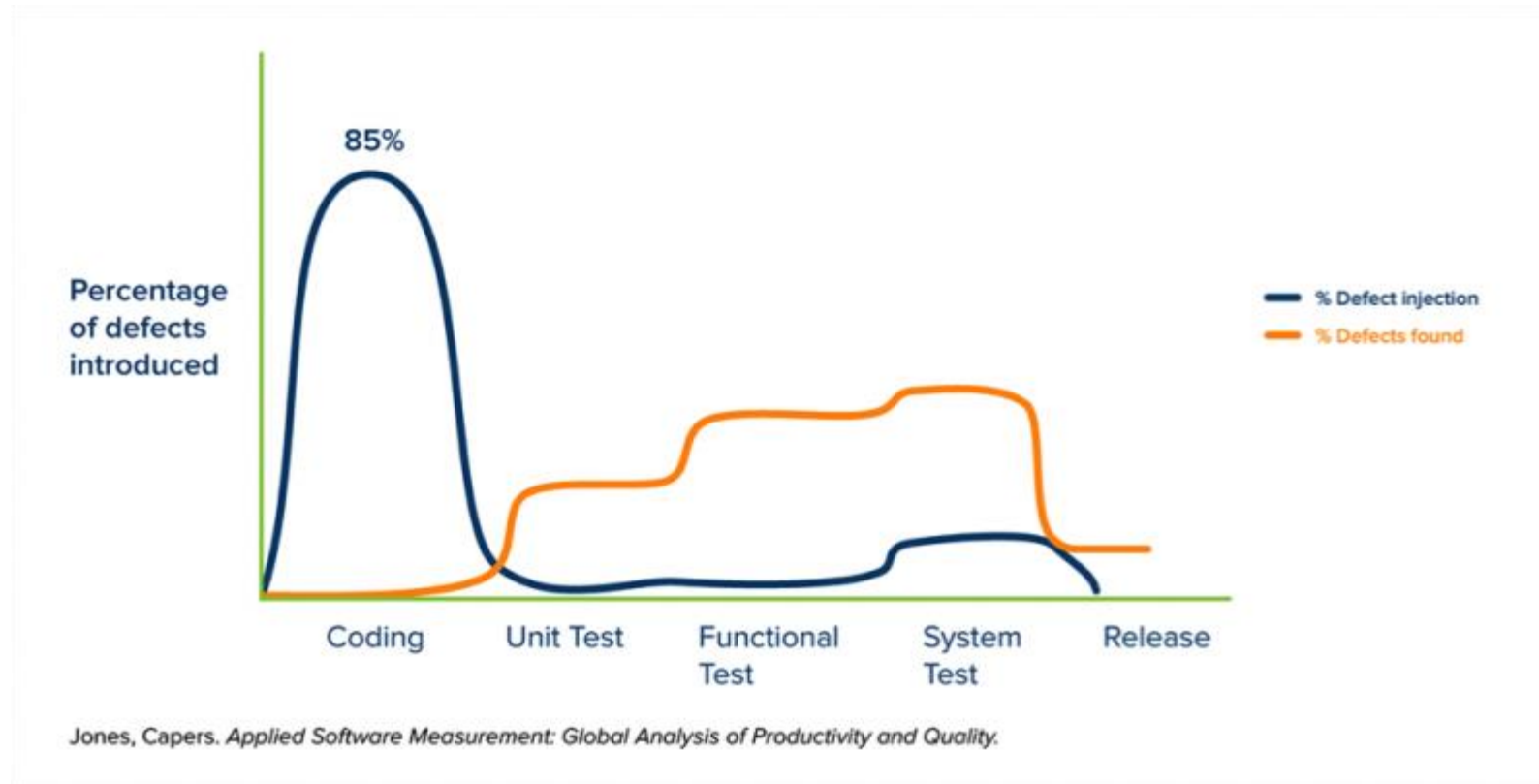
- 대부분의 버그는 코딩 단계에서 만들어진다.



결함 처리 비용

✓결함의 발견

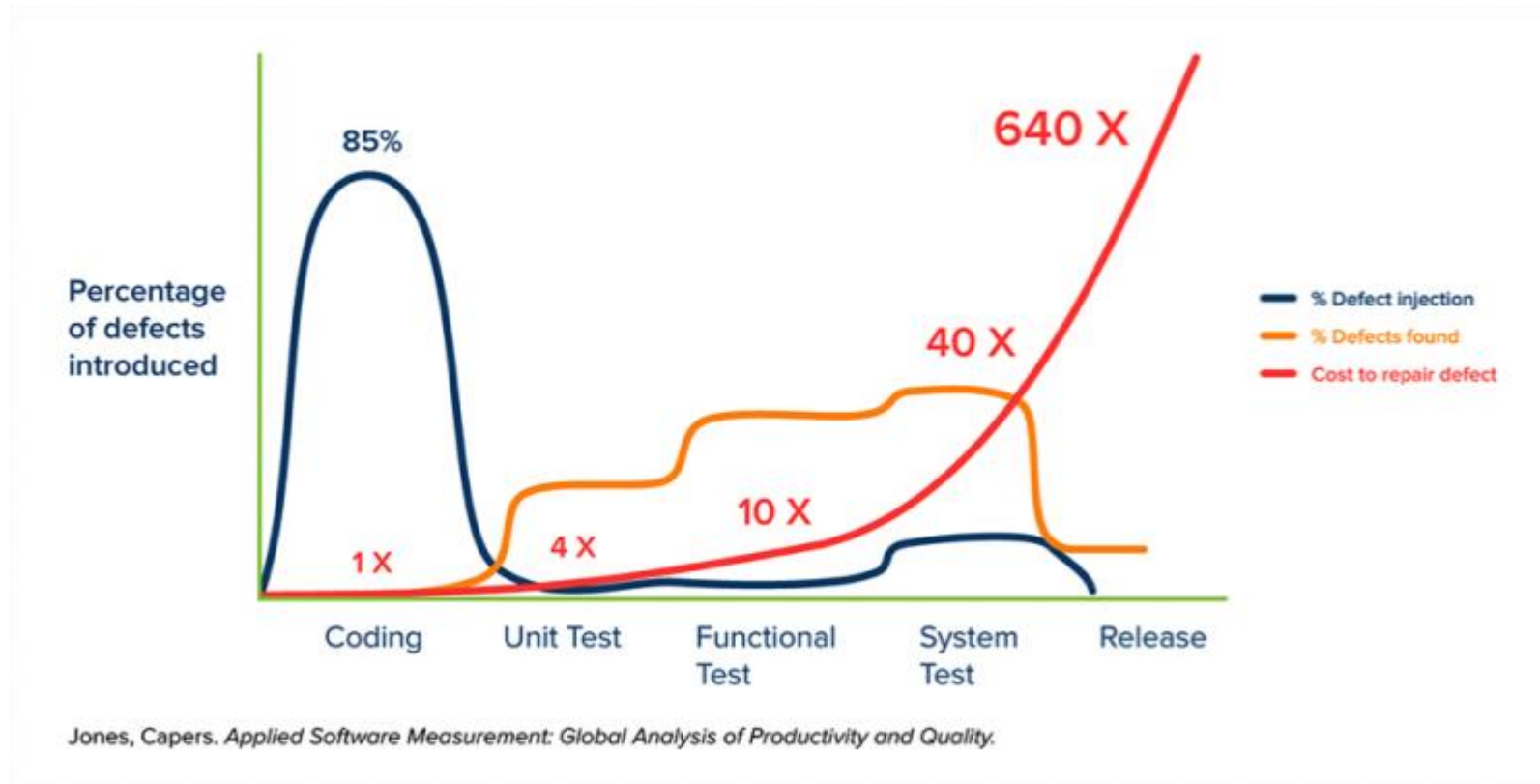
- 테스트를 시작할 때부터, 많은 결함이 발견된다.



결함 처리 비용

✓개발의 각 단계에서 결함을 수정하는데 드는 비용의 차이

- 개발이 진행될수록, 결함이 발견될수록 비용이 급격히 증가

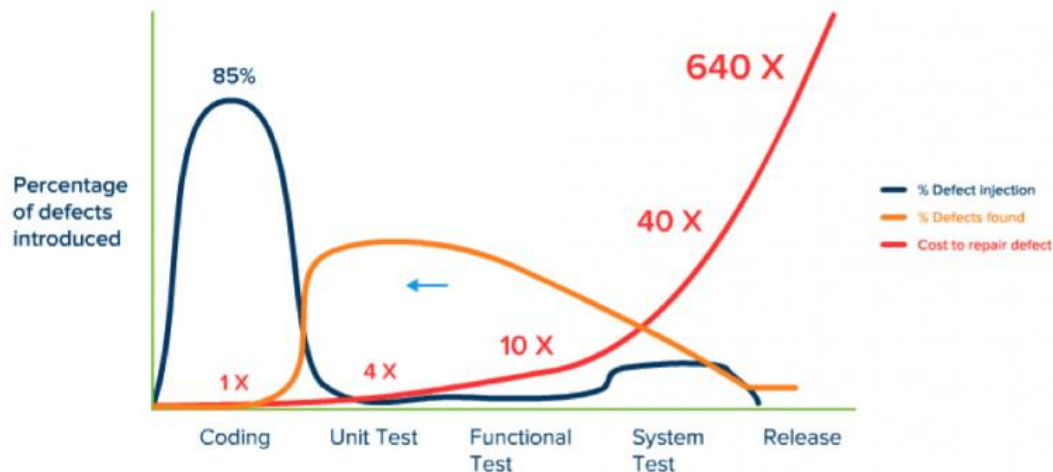


결함 처리 비용

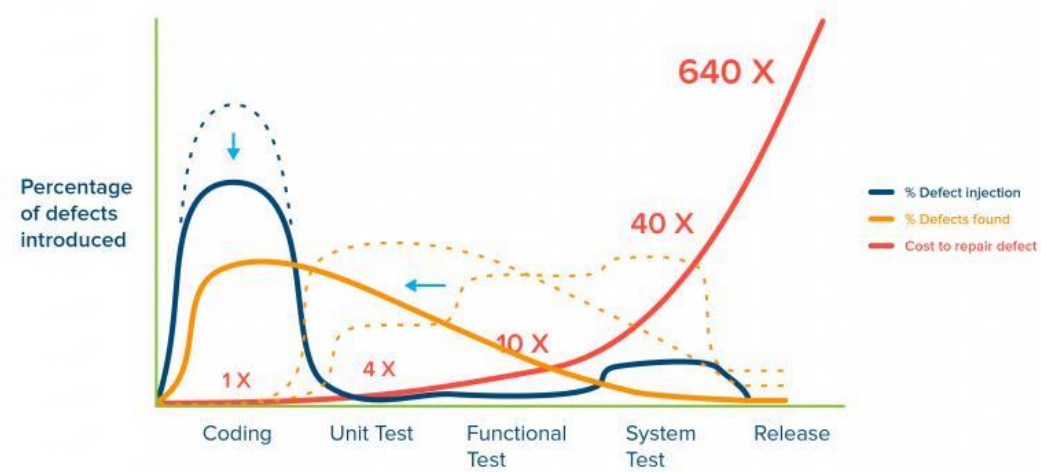
Shift Left을 지향해야한다.

✓테스트를 초기에 수행하면 일어나는 변화

- 코드의 문제를 일찍 발견할수록 그로 인한 영향이 줄어들고 문제를 해결하는 비용이 줄어든다



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.

Unit Test 작성 원칙 (F. I. R. S. T.) by 클린코드

Fast

- 테스트는 빨라야 한다.
- 테스트는 빨리 자주 돌릴 수 있어야 한다. 그래야 초반에 문제를 찾아낼 수 있다

Independent

- 각 테스트는 서로 의존하면 안 된다.
- 테스트가 실패할 때 원인 분석에는, 테스트 순서에 따른 Fail 의심이 없어야 한다.

Repeatable

- 테스트는 어떤 환경에서도 반복 가능해야 한다.
- 테스트가 실패할 때 원인 분석에는, 특정 시간 / 특정 환경에 따른 Fail 의심이 없어야 한다.

Self-Validating

- 테스트는 bool 값으로 결과를 내야 한다.
- Fail일 확률이 존재한다는 결과는, 도움이 되지 않는다.

Timely

- 테스트는 적시에 작성해야 한다
- 적시 = 개발하기 전
- 초반에 버그를 잡을 수 있도록 적시에 작성한다.